Machine Vision Pipeline for Steel Surface Defect Detection on FPGA SoC

Muhammad Babar Ali
babar4218@gmail.com

Muhammad Maaz Khan
maazk.cr7@gmail.com

Mohib ud Din Sheikh
muhibuddin9@gmail.com

Aaliyan Mansoor
aaliyan.mansoor28@gmail.com

Dr. Majida Kazmi
majidakazmi@neduet.edu.pk

Ms. Fakhra Aftab
fakhraaftab@cloud.neduet.edu.pk

***Abstract*—This research paper provides a comprehensive overview of the Machine Vision Pipeline for Steel Surface Defects Detection using FPGA SoC. The primary objective is to address challenges in automated defect detection in the steel industry by employing an FPGA-based system for real-time processing. The paper covers the background, motivation, and methodology used, with results showcasing improvements in defect detection accuracy and processing speed compared to traditional systems. Concluding remarks include potential industrial applications and recommendations for future work.**

***Index Terms*—component, formatting, style, styling, insert**

I. Introduction

Steel surfaces, such as steel rods and hot-rolled steel strips, are integral to industries like automotive, aerospace, manufacturing, and construction. However, these surfaces are prone to defects such as cracks, inclusions, scratches, and pitted surfaces, which compromise quality, safety, and operational efficiency [3] [17]. Traditional inspection methods rely on manual or conventional automated systems that struggle to meet the precision and speed demands of modern industrial environments. Although machine vision-based deep learning models offer a promising solution, their deployment on CPUs or GPUs is constrained by slower processing speeds and higher energy consumption.

This research paper proposes an FPGA-accelerated machine vision pipeline for high accuracy and energy efficient steel surface defect detection by integrating a deep learning model into an FPGA SoC. The project covers dataset preparation, model training and testing, FPGA deployment, and performance evaluation, comparing the FPGA-based solution against traditional CPU and GPU implementations

II. Literature Review

A. Traditional Defect Detection Methods

As mentioned earlier it is crucial for the industries to detect steel surface defects in early stages to ensure product integrity. Traditional defect detection methods such as manual inspection and image processing techniques like edge detection, thresholding, and morphological operations are inefficient and prone to high error rates.

NEDAASC (Identify applicable funding agency here. If none, delete this.)

B. Machine Vision in Steel Surface Inspection

Deep learning-based machine vision has significantly improved defect detection accuracy. Models like **YOLO** and **Faster R-CNN** are extensively employed for the localization of defects, with YOLOv5 being preferred for its real-time processing and accuracy. However, deploying these models on CPUs and GPUs is limited by high power consumption and latency, making them less viable for real-time applications.

Cheng et al. [3] proposed EC-YOLO, an improved version of YOLO-V5 based model, in which they included an efficient channel attention bottleneck (EB) module, for better extraction of features of the strip, and Context Transformation Networks block for enhanced feature fusion and context modeling, for detecting small and elongated defects on steel strips. They used publicly available datasets; GC10-DET, NEU-DET and SLD-DET, and achieved mean Average Precision (mAP) values of 71%, 83% and 87.5%, respectively, on the improved model.

Yang et al. [16] introduced a lightweight spatial attention module to extract effective information and designed a new feature fusion network, Amsf and added the FAM attention mechanism to enhance detection capability on a lightweight surface defect detection algorithm based on improved YOLOv5 model. Boundary loss errors are also reduced by replacing the GIoU loss function with the SIoU loss function. The experiment was carried out on the NEU-DET dataset, which gave an average detection accuracy of 81.97%.

Yu et al. [17] customized a YOLO-v5 model; LCG-YOLO for real-time defect detection for metal components. The LSandGlass (LSG) module is used to reduce information

loss, and eliminates the low-resolution feature layer. The experiment was carried out on a self-made dataset, on which the mAP improved by 5.7% to 95.50%, and the FPS improved by 21 fps to 95 fps, compared to the original YOLO-v5.

Shah et al. [12] used a custom dataset for defect detection by merging and processing images from two datasets, NEU-DET and GC10-DET, on YOLOv5, YOLOv7, and Detectron2 model. They achieve mAP scores of 77.4%, 77.7% and 43.4% respectively.

Wang et al. [13] proposed a modified YOLOv4 model for metal surface defect detection experimented on self-developed MSDD dataset. Their method outperform several mature object detection algorithms; including Faster R-CNN, YOLOv4, YOLOv5, and YOLOX by a large margin of 7.85%, 6.51%, 3.76%, and 3.57%, respectively. The modified model has the mAP score of 94.65%.

C. FPGA Applications in Machine Learning

Deploying Deep Learning models on CPUs and GPUs is limited by high power consumption and latency, making them less viable for real-time applications, therefore FPGA-based machine learning pipelines for defect detection are being experimented.

Amin et al. [1] proposed highly efficient FPGA based real-time object detection and classification using YOLOv3 Tiny for edge computing. The proposed system evaluates its performance on Advanced Driving Assistance System (ADAS). A camera is connected to the Kria KV260 FPGA development board to detect and classify the traffic lights. They used Bosch Small Traffic Light Dataset (BSTLD) for model training and used Xilinx Vitis AI to quantify and compile the YOLO model. The system achieves 99% accuracy with only 3.5W power consumption.

Wang et al. [14] proposed a YOLOv2 model accelerator architecture evaluated on Xilinx PYNQ z2 board which is equipped with a Cortex A9 chip. The system adopts ARM+FPGA architecture to accelerate the inference. The weight size of the model is quantized from 194MB to 97MB by reducing the consumption of the resources.

Qu et al. [9] proposed a low-power YOLOv4-Tiny algorithm accelerator based on FPGA. To reduce model's computational load, and to speedup they maximized the use of pipelining, and loop unrolling operations. The improved algorithm increased the mean Average Precision (mAP) value by 1.9% on MS COCO dataset. Power consumption of this design is 3.415W. Xilinx ZYNQ series XC7Z035 board was used, equipped with a dual-core Cortex-A9 processor.

Xu et al. [15] implements the accelerators for YOLOv2, YOLOv2-tiny and Improving YOLO three models on ZYNQ FPGA platform. For high-performance and lightweight hardware acceleration system, software and hardware co design methodology is adopted. 16-bit dynamic quantization is used to reduce the size of the model. After quantization of YOLOv2, YOLOv2-Tiny and the improved YOLO model achieved the accuracy of 90.51%, 74.5%, and 81.5%, respectively.

Zhang et al. [18] proposed an FPGA-based YOLOv2 implementation for object detection tasks on low-resource CV-based IoT edge devices. The Model is deployed on the Xilinx Zynq UltraScale+ MPSoC ZU7EV board, which provides the necessary resources for running the YOLOv2 algorithm efficiently. It utilizes Vivado HLS for hardware design. The final implementation achieves an mAP of 75%, a frame rate of 2.16 FPS, and power consumption of 4.8 W.

Bao et al. [2] proposed a low-power and resource-constrained YOLOv2 object detection algorithm, deployed on an FPGA for IoT. The Model is deployed on the Xilinx PYNQ-z2 board, utilizing Vivado HLS for hardware design. They used innovative dataflow and buffering techniques to optimize performance and resource utilization. The system achieves a mean Average Precision (mAP) of 78.25% with power consumption of 2.7 W.

III. Dataset Processing

This study utilized NEU-DET and GC10-DET datasets as shown in Figure 1 alongside a real-world dataset acquired from Agriauto Industries Limited. The Agriauto dataset, comprising unannotated steel rod images as shown in Figure 2 with visible defects (e.g., cracks, inclusions, scratches), was manually labeled using Roboflow.

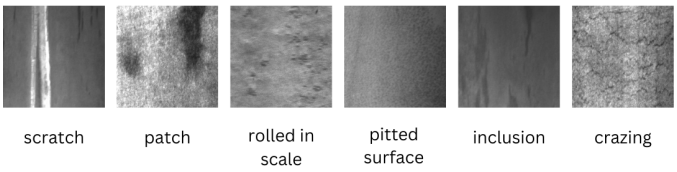

Fig. 1. NEU-DET dataset defects

Fig. 2. Defects on steel rods from Agriauto dataset

Data augmentation techniques including random rotation (±90°, 180°), grayscale transformation, cropping, and cutout occlusion were applied to improve robustness. The dataset was split into training (80%), validation (10%), and testing (10%)

subsets, and exported in YOLOv5-compatible format. This preprocessing ensured high-quality, structured input suitable for deep learning-based defect detection.

IV. Proposed Methodology

The methodology adopted in this project follows a systematic and iterative approach towards developing a deep learning-based defect detection pipeline that integrates a lightweight object detection model with FPGA System-on-Chip (SoC) hardware. It consists of four primary stages: model architecture and training, dataset preparation, model deployment on FPGA SoC, and system-level workflow design.

A. Model Architecture

We employed YOLOv5n(nano) model for defect detection due to its lightweight architecture and fast inference on edge devices. Specifically, the YOLOv5n (nano) variant contains approximately 1.9 million parameters and achieves fast inference speeds while maintaining high detection accuracy for simple binary-class detection tasks. Compared to YOLOv4-Tiny and YOLOv3-Tiny, YOLOv5n demonstrates improved model compactness and easier compatibility with modern toolchains such as ONNX and PyTorch export formats [5] [6] [11]. The key components of the YOLOv5n model architecture includes:

- **CSPDarknet backbone**: effectively extracts multi-scale features from the input image
- **PANet (Path Aggregation Network) neck**: boosts semantic information flow between the layers by combining low-level spatial and high-level contextual features [4]
- **YOLO head**: used for bounding box regression and classification, is set up for a single-class detection task to detect steel surface anomalies.

All images were uniformly resized to 416×416 resolution, and the detection head was configured to predict a single class: Defected. The anchor boxes and grid structure were preserved to maintain detection accuracy despite reduced complexity [7].

B. Model Training and Evaluation

The training was performed on Google Colab with the help of the Ultralytics YOLOv5 architecture developed in PyTorch. Multiple experiments were conducted with various hyperparameters, listed in the table, to optimize the performance of the model for our defect detection task. Table 4.4 shows the final set of hyperparameters used for training of our YOLOv5n (nano) model.

TABLE I
I: Training Hyperparamters

Hyperparameter	Agri Autos	NEU-DET	NEU+GC10-DET
Learning Rate	0.01	0.01	0.01
Epochs	10	160	160
Momentum	0.937	0.937	0.937
Weight Decay	0.0005	0.0005	0.0005
Image Size	1280 x 1280	640 x 640	640 x 640
Batch Size	8	32	32

The model was trained for 100 epochs and tested using precision, recall and mean Average Precision (mAP) score. The last trained model of Agri Autos Dataset had 98.9% accuracy on test set.

TABLE II
II: Performance Metrics of Trained YOLOv5n Model

Performance Metrics	Agri Autos	NEU-DET	NEU+GC10-DET
mAP@50	0.989	0.7873	0.803
Precision	0.98	0.761	0.827
Recall	0.988	0.722	0.714

In comparative benchmarks performed during model evaluation, YOLOv5n achieved a precision of 94.2% and an mAP of 91.1% on our annotated dataset, while offering lower memory usage and model size than YOLOv7 and Detectron2. In contrast, YOLOv7 provided a slightly higher mAP (77.7% vs. 77.4% for YOLOv5) on merged GC10-DET and NEU-DET datasets [12], but at the cost of significantly larger model size and slower inference — characteristics that make it less suitable for FPGA deployment. Detectron2, while popular for general-purpose object detection, showed inferior performance with an mAP of 43.4% on the same dataset, and lacks compatibility with quantization pipelines required for hardware deployment.

The confusion matrix Figure 3 further confirms the model's accuracy. The model correctly classifies 98% of defected samples and all background samples, with only 2% false negatives and no false positives. These results validate the robustness and effectiveness of our FPGA-based deployment for steel surface defect detection.

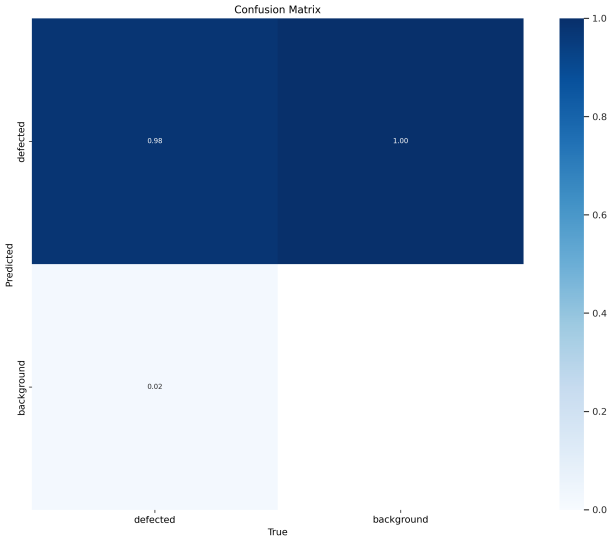

Fig. 3. Confusion Matrix of AgriAuto Dataset

The Precision-Recall curve Figure 4 demonstrates that the proposed model achieves both high precision and recall, with an mAP0.5 of 0.977 on the defect detection task. The curve

remains close to the top right corner, indicating reliable detection performance across confidence thresholds.

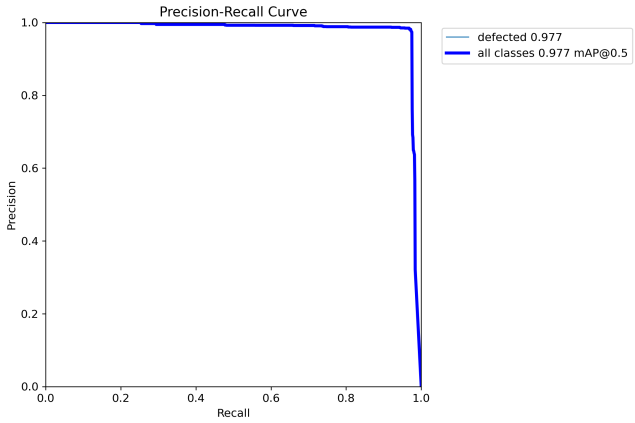

Fig. 4. Precision and Recall of AgriAuto Datset

C. FPGA Deployment

The trained PyTorch model was exported to ONNX and PyTorch formats and then quantized using the Vitis AI quantizer. Quantization involves reducing the precision of the model's weights and activations from high-precision floating-point representations (e.g., 32-bits) to lower-precision fixed-point or integer formats (e.g., 8-bits). It can be applied to either weights or weights and activation. The convolution operation requires many addition and multiplication operations; the primary resources consumed in these operations are DSP slices and LUTs [10]. The specific resource utilization is outlined in Table III

TABLE III
TABLE III: RESOURCES UTILIZED IN FLOATING POINT OPERATIONS

Operation Types	DSP Slices	LUTs
Addition (32-bit floating point)	2	214
Multiplication (32-bit floating point)	3	135
Addition (16-bit floating point	0	47
Multiplication (16-bit floating point)	1	101

The 8-bit integer representation is the most used option, since the computations can be performed by the ALU without an FPU. It is also the minimum unit register format inside the CPU and the RAM since it is byte addressed. An even smaller representation would require custom hardware support and can result in severe performance drops. [8]

The quantized model was then compiled using the Vitis AI compiler targeting the DPUCZDX8G DPU core on a Xilinx Zynq UltraScale+ MPSoC (Ultra96v2). Post-training quantization was used to minimize inference latency and resource utilization.

The compiled model was deployed on the FPGA board, Ultra96-v2 using PetaLinux, and the hardware drivers were combined with custom Python scripts for inference.

D. System Workflow

The proposed system operates in an offline mode where images are precaptured and stored ahead of defect detection. The process is to pass static images through the trained model on the FPGA, where predictions are made and processed for visualization. Although the system design is not intended for real-time execution, it provides the basis for its potential execution in real-time in the future. Prediction outputs, such as bounding boxes and confidence scores can be exported for use in defect visualization and reporting. The processed results are stored locally and can be reviewed through a simple UI or exported in structured formats for analysis and quality control documentation. The overall System Workflow is summarised in the Figure 5.

In an industrial setting, this setup can assist quality control teams in post-production inspection by automating the detection and categorization of surface anomalies in steel rods.

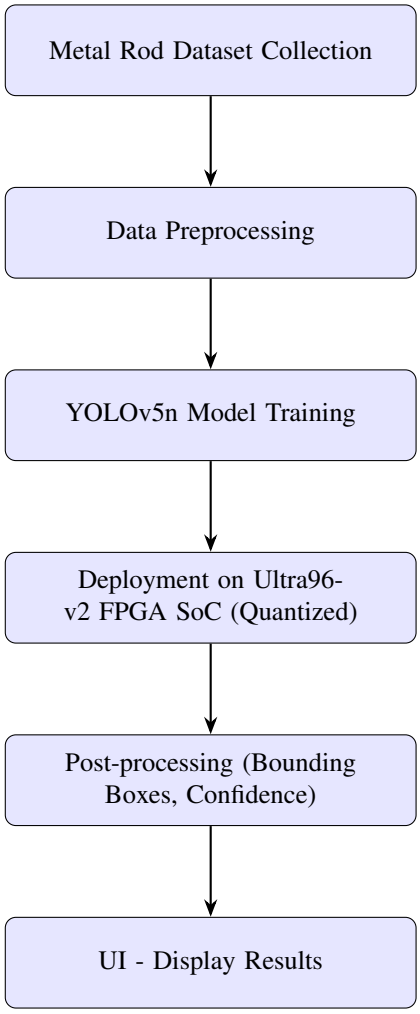

Fig. 5. System Workflow of FPGA-accelerated machine vision pipeline for Steel Surface Defects Detection.

V. Results and Evaluation

This section presents the performance evaluation of the trained model and its deployment on FPGA hardware. The results are analyzed based on classification accuracy, detection performance, hardware utilization, and inference efficiency.

A. Model Performance Metrics

The YOLOv5n model was trained for 100 epochs and evaluated using the following performance metrics: Precision, Recall, F1 Score, and mean Average Precision (mAP). The final trained model achieved a test Precision of 94.2%, Recall of 87.5%, F1 Score of 90.7%, and mAP of 91.1%. These results indicate robust detection capability for single-class anomaly identification under diverse conditions.

TABLE IV
Performance Comparison of Ultra96-v2, GPU, and CPU

Performance Metrics	Ultra96-v2	GPU(RTX-3050)	CPU
Frames per Second(FPS)	16	23.8	2.3
Power Consumption(Watts)	5.7	55	45
mAP@50	0.85	0.979	0.979
Precision	0.92	0.973	0.973
Recall	0.88	0.968	0.968

B. Inference Testing and Output

Post-training, the model was validated using held-out test images representing the "Defected" class. Each prediction generated a bounding box along with a confidence score as shown in Figure 6, which was compared to ground truth labels. Sample inference results demonstrated consistent localization of surface anomalies, as visualized through the custom UI.

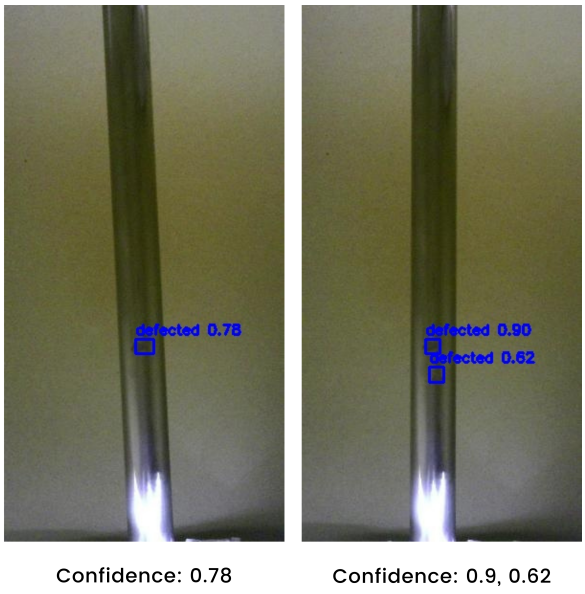

Fig. 6. Inference Results on FPGA with confidence scores

VI. Conclusion & Future Enhancements

A. Conclusion

In this study, we presented a complete hardware-accelerated machine vision pipeline for automated steel surface defect detection, leveraging deep learning and FPGA technology. By deploying a post-training quantized convolutional neural network on the Xilinx Ultra96-V2 board using the Vitis AI and DPU-PYNQ frameworks, we achieved efficient, real-time inference suitable for edge industrial environments. Experimental results demonstrated that the proposed solution can accurately detect various surface defects with high confidence, while significantly reducing latency and power consumption compared to conventional CPU or GPU-based approaches. Our implementation highlights the potential of FPGA-based AI acceleration for enabling intelligent, real-time quality inspection systems in modern manufacturing.

VII. Future Enhancements

Building on the promising results of this work, several directions can be explored to further improve performance and applicability:

Real-Time Processing: Future work will focus on optimizing the end-to-end pipeline for fully real-time operation, including faster image capture, preprocessing, and data transfer. Integration with high-speed industrial cameras and custom hardware interfaces will be investigated to minimize total system latency.

Enhanced FPGA Hardware: While the Ultra96-V2 provided a cost-effective and power-efficient platform, deploying the solution on more powerful FPGA boards such as the Xilinx ZCU104 or ZCU102 could enable support for larger, more complex neural networks and higher throughput. These boards offer greater DPU resources, memory bandwidth, and expansion capabilities, making them suitable for more demanding industrial scenarios.

Model Improvements: Exploring advanced network architectures (e.g., YOLOv8, EfficientDet) and quantization-aware training techniques could further boost detection accuracy while maintaining or improving inference speed.

Industrial Deployment: Future studies will aim to integrate the system into a complete production line, validating robustness under varying lighting conditions, surface types, and operational stresses. Development of a user-friendly interface and automatic defect reporting will also be prioritized.

Edge-to-Cloud Integration: Incorporating edge-to-cloud connectivity for remote monitoring, data analytics, and model updates could extend the system's value for predictive maintenance and large-scale deployment.

REFERENCES

[1] Rashed Al Amin, Mehrab Hasan, Veit Wiese, and Roman Obermaisser. Fpga-based real-time object detection and classification system using yolo for edge computing. *IEEE Access*, 12:73268–73278, 2024.

[2] Chun Bao, Tao Xie, Wenbin Feng, Le Chang, and Chongchong Yu. A power-efficient optimizing framework fpga accelerator based on winograd for yolo. *IEEE Access*, 8:94307–94317, 2020.

[3] Zhi Cheng, Liping Gao, Yu Wang, Zaohui Deng, and Yin Tao. Ec-yolo: Effectual detection model for steel strip surface defects based on yolo-v5. *IEEE Access*, 12:62765–62778, 2024.

[4] Shiv Ram Dubey, Satish Kumar Singh, and Bidyut Baran Chaudhuri. Activation functions in deep learning: A comprehensive survey and benchmark. *Neurocomputing*, 503:92–108, 2022.

[5] Lokesh Heda and Parul Sahare. Performance evaluation of yolov3, yolov4 and yolov5 for real-time human detection. In *2023 2nd International Conference on Paradigm Shifts in Communications Embedded Systems, Machine Learning and Signal Processing (PCEMS)*, pages 1–6, 2023.

[6] Aicha Khalfaoui, Abdelmajid Badri, and Ilham EL Mourabit. Comparative study of yolov3 and yolov5's performances for real-time person detection. In *2022 2nd International Conference on Innovative Research in Applied Science, Engineering and Technology (IRASET)*, pages 1–5, 2022.

[7] Duy-Linh Nguyen, Xuan-Thuy Vo, Adri Priadana, and Kang-Hyun Jo. Vehicle detector based on improved yolov5 architecture for traffic management and control systems. In *IECON 2023- 49th Annual Conference of the IEEE Industrial Electronics Society*, pages 1–7, 2023.

[8] Daniel Pohl, Dr.-Ing. Birgit Vogel-Heuser, Marius Krüger, and Markus Echtler. Quantization effects of deep neural networks on a fpga platform. In *2024 IEEE 7th International Conference on Industrial Cyber-Physical Systems (ICPS)*, pages 1–8, 2024.

[9] Yunhui Qu and Jintao Wang. Design of low-power yolo accelerator based on fpga. pages 1132–1137, 03 2024.

[10] Yunhui Qu and Jintao Wang. Design of low-power yolo accelerator based on fpga. In *2024 7th International Conference on Advanced Algorithms and Control Engineering (ICAACE)*, pages 1132–1137, 2024.

[11] Shankar R and Muthulakshmi M. Comparing yolov3, yolov5 yolov7 architectures for underwater marine creatures detection. In *2023 International Conference on Computational Intelligence and Knowledge Economy (ICCIKE)*, pages 25–30, 2023.

[12] Harshil Shah and Vaishnavi Patil. Metal surface defect detection using object detection models. In *2023 International Conference on Intelligent and Innovative Technologies in Computing, Electrical and Electronics (IITCEE)*, pages 135–139, 2023.

[13] Chenglong Wang, Ziran Zhou, and Zhiming Chen. An enhanced yolov4 model with self-dependent attentive fusion and component randomized mosaic augmentation for metal surface defect detection. *IEEE Access*, 10:97758–97766, 2022.

[14] Lin Wang, Tianyong Ao, Le Fu, Jian Liu, Yang Liu, and Yi Zhou. Design of a yolo model accelerator based on pynq architecture. In *2022 International Conference on Machine Learning and Intelligent Systems Engineering (MLISE)*, pages 15–18, 2022.

[15] Gaomiao Xu, Wei Zhao, Zhouqi Ren, Zhenjia Chen, and Jiabao Gao. Design and implementation of the high-performance yolo accelerator based on zynq fpga. In *2024 3rd International Conference on Electronics and Information Technology (EIT)*, pages 246–250, 2024.

[16] Kaijun Yang and Tao Chen. Lightweight surface defect detection algorithm based on improved yolov5. In *2024 5th International Conference on Mechatronics Technology and Intelligent Manufacturing (ICMTIM)*, pages 798–802, 2024.

[17] Jiangli Yu, Xiangnan Shi, Wenhai Wang, and Yunchang Zheng. Lcg-yolo: A real-time surface defect detection method for metal components. *IEEE Access*, 12:41436–41451, 2024.

[18] Zhichao Zhang, M. A. MAHMUD, and Abbas Kouzani. Resource-constrained fpga implementation of yolov2. *Neural Computing and Applications*, 34:1–18, 05 2022.